PLP - Segundo Parcial - 1er cuatrimestre de 2025

#Orden	Nro. Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (`B`) y uno regular (`R`), y se promociona con al menos dos ejercicios muy bien (`MB`) y uno bien (`B`). Es posible obtener una aprobación condicional con un ejercicio muy bien (`MB`), uno bien (`B`) y uno insuficiente (`I`), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: `-`, `I`, `R`, `B`, `MB`, `E`. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (`!`) ni predicados de alto orden como `setof`, con la única excepción de `not`. No está permitido usar macros que colapsen cláusulas.

a) Definir el predicado `esRotación(+L,+R)` que es verdadero cuando `R` es una rotación de `L`. Una lista `R` es una rotación de una lista `L` si contiene los mismos elementos en el mismo orden relativo, pero comenzando desde una posición diferente dentro de `L`, recorriendo la lista circularmente. Por ejemplo:

```
?- esRotacion([1,2,3,4], [3,4,1,2]).
true.
?- esRotacion([a,b,c], [c,a,b]).
true.
?- esRotacion([a,b,c], [c,b,a]).
false.
```

b) La *secuencia de Collatz* se construye con la siguiente regla sobre un número natural: si n es par, el siguiente número es $n/2$; si n es impar, el siguiente número es $3n+1$. La secuencia termina cuando se llega al número 1 (aunque la conjetura dice que siempre se llega a 1, esto no ha sido probado para todos los naturales).
Definir el predicado `collatz(+N,-S)` que instancia en `S` todos los números pertenecientes a la secuencia de Collatz a partir del número `N`. Por ejemplo:

```
?- collatz(6,S).
S = 6 ;
S = 3 ;
S = 10 ;
S = 5 ;
S = 16 ;
```

```
S = 8 ;
S = 4 ;
S = 2 ;
S = 1 ;
false.
```

c) ¿El predicado anterior es reversible en `S`? (pueden asumir que se utilizarán números bajos por lo que la secuencia es siempre finita). Justificar brevemente.

d) Definir el predicado `collatzMayor(+N,-M)` que es verdadero cuando `M` es el número más grande de la secuencia de Collatz a partir del número `N`. Por ejemplo:

```
?- collatzMayor(6,M).
M = 16 ;
false.
```

Ejercicio 2 - Resolución

a. Dada la siguiente base de conocimientos en Prolog:

```
member(X,[X|_]).
member(X,[Y|XS]) :- member(X,XS).

esSublista(_,[]).
esSublista(L,[X|XS]) :- member(X,L), esSublista(L,XS).
```

Explicar **brevemente** con palabras qué sucede al realizar la siguiente consulta:

```
esSublista([a,b],R), member(b,R).
```

b. Expresar la base de conocimientos y la consulta anterior como fórmulas lógicas.

c. Pasar las fórmulas a forma clausal y encontrar una solución a la consulta utilizando resolución.

d. La resolución utilizada en el punto anterior ¿fue SLD? Justificar.

e. ¿Hay alguna diferencia entre su resolución y en cómo resuelve Prolog la consulta? Si es así, explicar cuál.

Ejercicio 3 - Inferencia, Deducción Natural y POO

a) Consideremos el Cálculo Lambda tipado extendido con árboles binarios con información en las hojas de la siguiente manera:

$$\tau ::= \cdots \mid \texttt{AIH}(\tau)$$
$$M ::= \cdots \mid \texttt{Hoja}(M) \mid \texttt{Bin}(M, M) \mid \mathsf{foldAIH}\ M \triangleright\ \mathsf{Hoja}(x) \rightsquigarrow M;\ \mathsf{Bin}(i, d) \rightsquigarrow M$$

- $\texttt{AIH}(\tau)$ es el tipo de los árboles con información en las hojas de tipo τ.
- $\texttt{Hoja}(M)$ es un árbol compuesto por una única hoja con información M.
- $\texttt{Bin}(M_1, M_2)$ es un árbol compuesto por dos subárboles M_1 y M_2.
- $\mathsf{foldAIH}\ M_1 \triangleright\ \mathsf{Hoja}(x) \rightsquigarrow M_2;\ \mathsf{Bin}(i, d) \rightsquigarrow M_3$ representa el esquema de recursión estructural sobre árboles con los casos de hoja (en donde la variable x se liga con el valor de la hoja y puede aparecer libre en M_2) y bin (que ligará las variables i y d al resultado de la recursión sobre sus subárboles y pueden aparecer libres en M_3).

Se extiende también el algoritmo de inferencia de la siguiente manera:

$$\mathcal{I}(\Gamma \mid \texttt{Hoja}(M)) = (\texttt{AIH}(\tau) \mid E) \qquad \text{donde } \mathcal{I}(\Gamma \mid M) = (\tau \mid E)$$

$$\begin{aligned}
&\mathcal{I}(\Gamma \mid \texttt{Bin}(M_1, M_2)) = (\tau_1 \mid \{\tau_2 \stackrel{?}{=} \texttt{AIH}(X_h), \tau_1 \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2) \\
&\quad \text{donde} \quad \mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1) \\
&\qquad\qquad\ \ \mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2) \\
&\qquad\qquad\ \ X_h \text{ variable fresca}
\end{aligned}$$

$$\begin{aligned}
&\mathcal{I}(\Gamma \mid \mathsf{foldAIH}\ M_1 \triangleright\ \mathsf{Hoja}(x) \rightsquigarrow M_2;\ \mathsf{Bin}(i, d) \rightsquigarrow M_3) = \\
&(\tau_2 \mid \{\tau_1 \stackrel{?}{=} \texttt{AIH}(X_x), \tau_2 \stackrel{?}{=} \tau_3, X_i \stackrel{?}{=} \tau_2, X_d \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2 \cup E_3) \\
&\quad \text{donde} \quad \mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1) \\
&\qquad\qquad\ \ \mathcal{I}(\Gamma, x : X_x \mid M_2) = (\tau_2 \mid E_2) \\
&\qquad\qquad\ \ \mathcal{I}(\Gamma, i : X_i, d : X_d \mid M_3) = (\tau_3 \mid E_3) \\
&\qquad\qquad\ \ X_x, X_i \text{ y } X_d \text{ variables frescas}
\end{aligned}$$

Usar el algoritmo $\mathcal{I}$ para inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

$$\mathsf{foldAIH}\ \texttt{Bin}(\texttt{Hoja}(\underline{1}), \texttt{Hoja}(x)) \triangleright\ \mathsf{Hoja}(x) \rightsquigarrow \mathit{isZero}(x);\ \mathsf{Bin}(x, y) \rightsquigarrow x$$

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$((\exists X.\neg P(X)) \wedge (\forall X.\neg R(X))) \Rightarrow \exists X.\neg(P(X) \wedge Q(X))$$

c) Implementar el método `#twice` cuyo objeto receptor es un bloque de un parámetro y devuelve un nuevo bloque que aplica el original dos veces. Indicar también dónde debería ir definido el método. Por ejemplo, la siguiente ejecución evalúa a 12:

```
| double doubleTwice |
double := [:x | x * 2].
doubleTwice := double twice.
doubleTwice value: 3.
```